

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 - Redes

Sección 20

Ing. Jorge Andrés Yass Coy



Laboratorio 2 - Esquemas de detección y corrección de errores

Bryan Alberto Martínez Orellana 23542

Adriana Sophia Palacios Contreras 23044

GUATEMALA
31 de julio de 2026

Descripción de la Práctica

La práctica integra los conceptos de arquitecturas por capas, desarrollo de protocolos y control de errores en la capa de enlace. Para aplicarlos, se implementó el protocolo de interacción entre un usuario y un cajero automático, utilizando una capa de aplicación para manejar el flujo bancario, una capa de presentación para convertir los mensajes a binario mediante ASCII, una capa de enlace para aplicar la detección o corrección de errores, una capa de ruido para introducir flips y una capa de transmisión para comunicar el cliente y el servidor mediante sockets (Yass, 2026a, 2026b).

El cliente se desarrolló en Python y el servidor en Java. En la capa de enlace se implementó Hamming (12,8) para corregir errores y CRC-32 para detectarlos. Finalmente, se realizaron pruebas con diferentes tamaños de mensajes y probabilidades de ruido, utilizando de forma acumulativa el texto de las páginas 13 a la 18 del capítulo 783 del manga One Piece (cnet128, 2015). Con esto se buscó comparar el tiempo de transmisión, la redundancia y la respuesta de ambos algoritmos ante los errores introducidos.

Resultados

Etapa de protocolo bancario

En esta etapa se implementó el protocolo bancario planteado durante las primeras semanas del curso. Primero, el cliente y el servidor realizaban un intercambio inicial para confirmar la conexión y seleccionar el algoritmo que se utilizaría durante la sesión. Luego, antes de cada mensaje protegido, el usuario podía indicar la cantidad esperada de flips en la trama. La explicación completa del ruido se mostraba únicamente la primera vez, aunque esta capa continuaba aplicándose en todos los envíos.

Posteriormente, se solicitaba el número de tarjeta y el PIN para autenticar al usuario. Si alguna credencial era incorrecta, la sesión se cerraba; de lo contrario, se mostraba un menú para consultar el saldo o retirar dinero. En el caso del retiro, el cliente validaba que el monto fuera un entero mayor que cero y el servidor verificaba que existieran fondos suficientes, permitiendo ingresar otro monto

cuando fuera necesario. Finalmente, el servidor actualizaba el saldo y enviaba la confirmación al cliente.

A continuación, se presenta una ejecución del protocolo bancario utilizando Hamming (12,8), en la cual se introdujo ruido durante algunos de los mensajes protegidos para poder observar el proceso de corrección y la continuidad de la operación.

Seleccione el algoritmo:

1. Hamming
 2. CRC-32
- > 1

=== INICIO DE TRANSACCIÓN ===

¡HAS SIDO HACKEADO!

=== SIMULACIÓN DE RUIDO ===

Cada mensaje protegido pasa por esta capa antes de enviarse.

El valor representa cuántos bits cambiarán en promedio.

La cantidad exacta puede variar porque cada bit se evalúa independientemente.

Hamming trabaja con bloques de 12 bits y puede corregir como máximo un error por bloque.

--- ENVÍO PROTEGIDO: START_TRANSACTION ---

Trama: 216 bits | Algoritmo: HAMMING

Flips promedio entre 0 y 216 [Enter = 0]:

> 2

HAMMING: se detectó un síndrome y se aplicó la corrección. La operación continuará.

=== AUTENTICACIÓN ===

Número de tarjeta: 123456

--- ENVÍO PROTEGIDO: CARD ---

Trama: 132 bits | Algoritmo: HAMMING

Flips promedio entre 0 y 132 [Enter = 2]:

> 0

PIN: 0507

--- ENVÍO PROTEGIDO: PIN ---

Trama: 96 bits | Algoritmo: HAMMING

Flips promedio entre 0 y 96 [Enter = 0]:

> 4

HAMMING: se detectó un síndrome y se aplicó la corrección. La operación continuará.

=== OPERACIÓN BANCARIA ===

Seleccione una operación:

1. Consultar saldo

2. Retirar dinero

> 2

--- ENVÍO PROTEGIDO: OPTION ---

Trama: 96 bits | Algoritmo: HAMMING

Flips promedio entre 0 y 96 [Enter = 4]:

> 0

=== RETIRO ===

Monto a retirar: 1200

--- ENVÍO PROTEGIDO: AMOUNT ---

Trama: 132 bits | Algoritmo: HAMMING

Flips promedio entre 0 y 132 [Enter = 0]:

> 0

Retiro realizado correctamente. Nuevo saldo: 8800

Sesión finalizada correctamente con HAMMING.

En esta ejecución, Hamming detectó y corrigió alteraciones durante el inicio de la transacción y el envío del PIN, permitiendo continuar con el protocolo. Finalmente, se realizó un retiro de 1200 y el saldo se actualizó correctamente de 10000 a 8800.

Etapa de pruebas

Para evaluar CRC-32 y Hamming (12,8), se utilizaron mensajes contruidos con el contenido de las páginas 13 a la 18 del capítulo 783 de One Piece. Primero se

transmitió únicamente la página 13 y, posteriormente, se agregó una página en cada etapa hasta formar un mensaje con las seis páginas. De esta forma, se buscó observar el comportamiento de los algoritmos conforme aumentaba el tamaño de la información.

Cada algoritmo se ejecutó por separado utilizando probabilidades de error de 0 %, 0.01 %, 0.1 % y 1 %. Para cada combinación se realizaron primero cinco ejecuciones de calentamiento, las cuales se descartaron para reducir la influencia del tiempo adicional que suele presentarse durante las primeras transmisiones, debido a la inicialización y estabilización de la comunicación y de los entornos de ejecución. Posteriormente, se registraron 100 repeticiones, obteniendo un total de 2400 pruebas para CRC-32 y 2400 para Hamming.

Durante cada prueba se registró el tamaño del mensaje original, la cantidad de bits transmitidos, la redundancia agregada, el porcentaje de sobrecarga, la cantidad real de flips, el tiempo de ida y vuelta y el resultado de la recuperación. Los datos de cada algoritmo se almacenaron en un archivo CSV y posteriormente se analizaron mediante un notebook de Jupyter utilizando Pandas, donde se calcularon medidas como la media, mediana, desviación estándar, mínimo y máximo, además de las tasas de detección o recuperación. Adicional, se utilizó ChatGPT, modelo GPT-5.6 Sol, como apoyo para revisión de redacción, aclaración de dudas en relación a conceptos, y como fuente para discutir decisiones (OpenAI, 2026).

A continuación, se presentan los datos estadísticos obtenidos durante las ejecuciones de CRC-32 y Hamming (12,8).

	pages	configured_error_probability_percent	ensayos	media_ms	mediana_ms	desviación_estándar_ms	mínimo_ms	máximo_ms
0	13	0.00	100	2.6835	2.6081	0.1871	2.4512	3.2688
1	13	0.01	100	2.6082	2.5980	0.0740	2.4360	3.0049
2	13	0.10	100	2.5955	2.5965	0.0429	2.4135	2.7532
3	13	1.00	100	2.6761	2.6076	0.3336	2.4344	4.9852
4	13-14	0.00	100	5.2623	5.2823	0.0822	4.9838	5.4103
5	13-14	0.01	100	5.2812	5.2752	0.0955	4.9800	5.5686
6	13-14	0.10	100	5.2238	5.2200	0.0769	5.0480	5.7050
7	13-14	1.00	100	5.1640	5.1674	0.0521	5.0178	5.2919
8	13-15	0.00	100	6.5777	6.5723	0.0675	6.4301	6.8220
9	13-15	0.01	100	6.4897	6.4674	0.0803	6.3805	6.9799
10	13-15	0.10	100	6.4985	6.4903	0.0815	6.3360	7.1492
11	13-15	1.00	100	6.4849	6.4615	0.1192	6.2171	7.1703
12	13-16	0.00	100	9.8468	9.7788	0.4400	9.5235	13.4052
13	13-16	0.01	100	9.8366	9.7933	0.1548	9.6408	10.5330
14	13-16	0.10	100	9.8021	9.7959	0.0910	9.6264	10.2175
15	13-16	1.00	100	9.6485	9.6487	0.0743	9.4130	9.8340
16	13-17	0.00	100	13.5188	13.4380	0.2880	13.2890	15.3029
17	13-17	0.01	100	13.4894	13.4548	0.1454	13.2828	14.0123
18	13-17	0.10	100	13.4694	13.4609	0.0852	13.3129	13.9863
19	13-17	1.00	100	13.2937	13.2761	0.1251	13.1230	14.2082
20	13-18	0.00	100	14.7751	14.5900	1.4925	14.4116	29.4527
21	13-18	0.01	100	14.6193	14.5957	0.1380	14.3886	15.3274
22	13-18	0.10	100	14.7974	14.6399	0.7898	14.3521	20.2704
23	13-18	1.00	100	14.3994	14.3699	0.1227	14.2280	14.8965

Tabla 1. Estadísticas del tiempo de transmisión utilizando Hamming (12,8).

	configured_error_probability_percent	status	cantidad	porcentaje
0	0.00	CLEAN	600	100.00
1	0.01	CLEAN	100	16.67
2	0.01	HAMMING_CORRECTED	497	82.83
3	0.01	HAMMING_RECOVERY_FAILED	3	0.50
4	0.10	HAMMING_CORRECTED	529	88.17
5	0.10	HAMMING_RECOVERY_FAILED	71	11.83
6	1.00	HAMMING_CORRECTED	5	0.83
7	1.00	HAMMING_RECOVERY_FAILED	595	99.17

Tabla 2. Distribución de los resultados obtenidos con Hamming (12,8).

	pages	payload_bits	serialized_data_bits	codeword_bits	redundancy_bits	overhead_percentage
0	13	4344	4496	6744	2248	50.0
1	13-14	9288	9440	14160	4720	50.0
2	13-15	10912	11064	16596	5532	50.0
3	13-16	17648	17800	26700	8900	50.0
4	13-17	23208	23360	35040	11680	50.0
5	13-18	25472	25624	38436	12812	50.0

Tabla 3. Redundancia y sobrecarga generadas por Hamming (12,8).

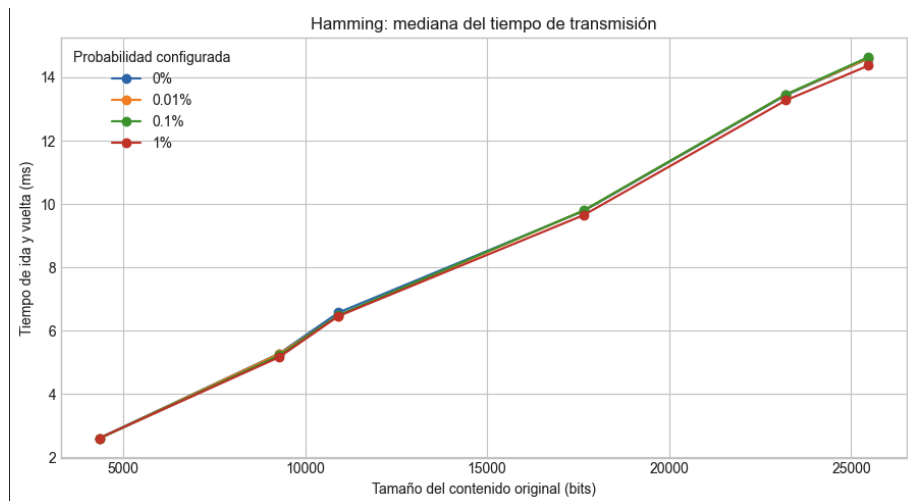


Figura 1. Mediana del tiempo de transmisión de Hamming (12,8) según el tamaño del mensaje.

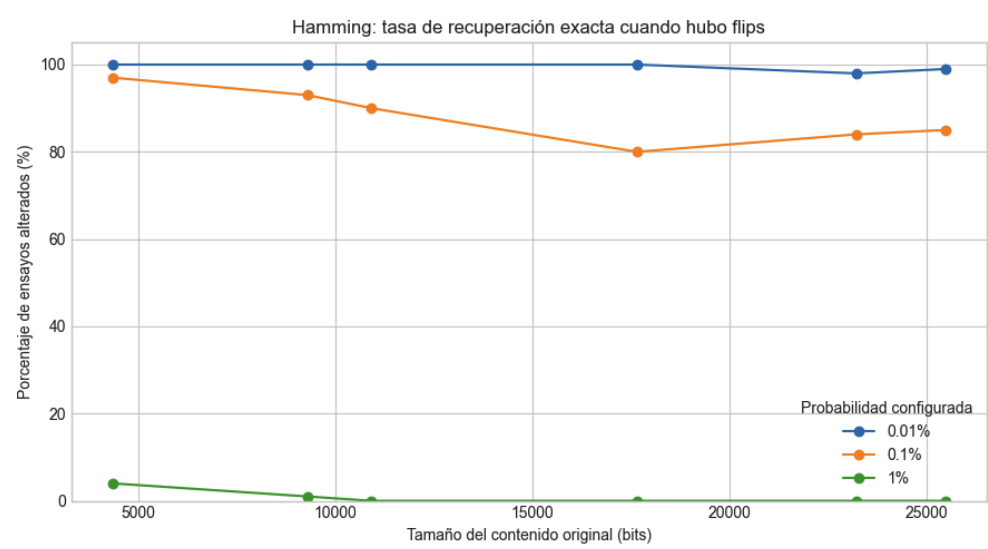


Figura 2. Tasa de recuperación de Hamming (12,8) en las transmisiones que presentaron flips.

	pages	configured_error_probability_percent	ensayos	media_ms	mediana_ms	desviación_estándar_ms	mínimo_ms	máximo_ms
0	13	0.00	100	4.3701	4.3550	0.0846	4.2248	4.7137
1	13	0.01	100	4.3719	4.3559	0.0946	4.2408	4.7488
2	13	0.10	100	4.4983	4.4945	0.0735	4.2758	4.7704
3	13	1.00	100	4.4817	4.4753	0.0582	4.3514	4.6372
4	13-14	0.00	100	9.3788	9.3703	0.1272	9.1280	9.9045
5	13-14	0.01	100	9.3619	9.3138	0.2236	9.0432	10.3076
6	13-14	0.10	100	9.3178	9.3003	0.1049	9.1386	9.6924
7	13-14	1.00	100	9.4330	9.3837	0.2283	9.0838	10.3821
8	13-15	0.00	100	12.2578	11.1891	5.8344	10.8303	65.5686
9	13-15	0.01	100	10.9122	10.8618	0.2906	10.4323	11.6792
10	13-15	0.10	100	10.8909	10.8310	0.2114	10.5059	11.4539
11	13-15	1.00	100	11.3113	10.8965	2.7738	10.6160	34.3362
12	13-16	0.00	100	18.4779	18.0804	2.5972	17.6438	43.3248
13	13-16	0.01	100	17.7906	17.5174	0.6456	17.1480	21.5216
14	13-16	0.10	100	18.5977	17.8091	4.2164	17.1645	53.5411
15	13-16	1.00	100	17.9940	17.6179	2.4192	17.2578	41.2962
16	13-17	0.00	100	24.1704	23.6553	2.7964	23.0447	50.4910
17	13-17	0.01	100	22.5538	22.4121	0.6238	22.0367	27.3392
18	13-17	0.10	100	22.9646	22.4777	2.4172	21.9895	39.8036
19	13-17	1.00	100	23.8850	23.1114	3.6866	22.2677	42.8863
20	13-18	0.00	100	26.7694	26.2925	2.7355	25.2254	45.3941
21	13-18	0.01	100	25.2492	24.7174	2.4401	24.0994	43.8868
22	13-18	0.10	100	25.6814	25.1015	3.3488	24.3788	45.5564
23	13-18	1.00	100	24.4171	24.3649	0.2946	24.0142	26.1677

Tabla 4. Estadísticas del tiempo de transmisión utilizando CRC-32.

	configured_error_probability_percent	status	cantidad	porcentaje
0	0.00	CLEAN	600	100.0
1	0.01	CLEAN	195	32.5
2	0.01	CRC_DETECTED	405	67.5
3	0.10	CRC_DETECTED	600	100.0
4	1.00	CRC_DETECTED	600	100.0

Tabla 5. Distribución de los resultados obtenidos con CRC-32.

	pages	payload_bits	serialized_data_bits	codeword_bits	redundancy_bits	overhead_percentage
0	13	4344	4496	4528	32	0.71
1	13-14	9288	9440	9472	32	0.34
2	13-15	10912	11064	11096	32	0.29
3	13-16	17648	17800	17832	32	0.18
4	13-17	23208	23360	23392	32	0.14
5	13-18	25472	25624	25656	32	0.12

Tabla 6. Redundancia y sobrecarga generadas por CRC-32.

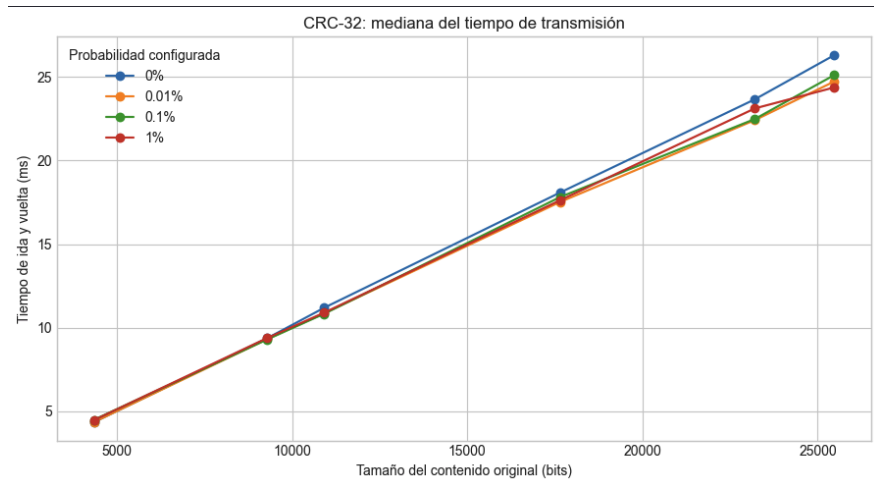


Figura 3. Mediana del tiempo de transmisión de CRC-32 según el tamaño del mensaje.

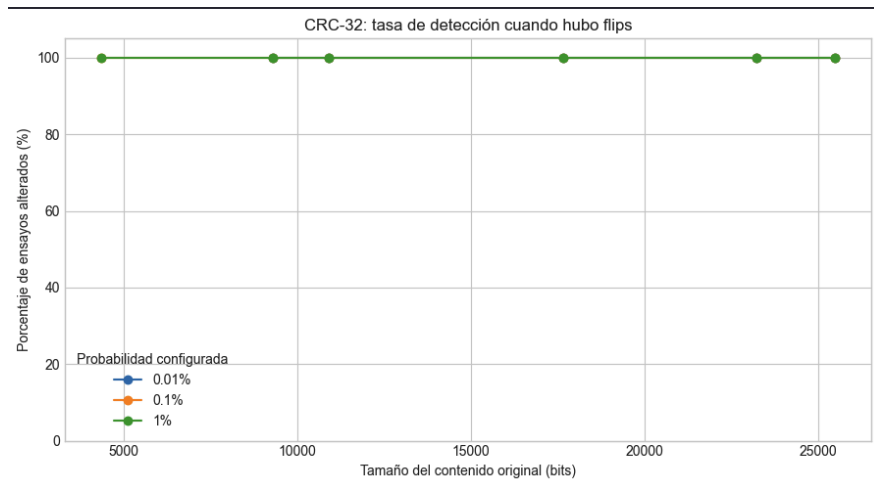


Figura 4. Tasa de detección de CRC-32 en las transmisiones que presentaron flips.

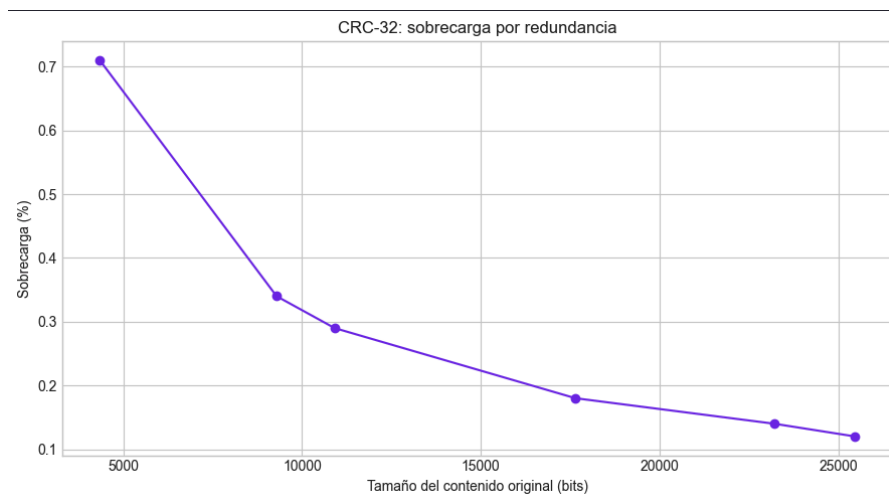


Figura 5. Porcentaje de sobrecarga generado por CRC-32 según el tamaño del mensaje.

Discusión

El desarrollo del protocolo permitió integrar varios de los conceptos trabajados durante el curso, principalmente la arquitectura por capas, la comunicación mediante sockets y los algoritmos de la capa de enlace. Uno de los aspectos que consideramos más relevantes fue comprobar que el protocolo era independiente del lenguaje utilizado, ya que el cliente se desarrolló en Python y el servidor en Java, pero ambos lograron comunicarse a partir de las mismas reglas. Los mensajes siguieron el formato COMANDO|parámetro, donde el parámetro solamente se incluía cuando era necesario, como al enviar el número de tarjeta o el monto de un retiro. Asimismo, la implementación permitió familiarizarnos con herramientas de Python como socket y argparse, y comprender con mayor claridad qué responsabilidad correspondía a cada capa (Yass, 2026b).

En cuanto a Hamming (12,8), la Tabla 1 y la Figura 1 muestran que el tiempo de transmisión aumentó principalmente con el tamaño del mensaje. Para una sola página, la mediana se mantuvo alrededor de 2.6 ms, mientras que para las seis páginas alcanzó aproximadamente 14.5 ms. Las líneas correspondientes a 0 %, 0.01 %, 0.1 % y 1 % permanecieron bastante cercanas, por lo que la probabilidad de error no tuvo un impacto sustancial sobre el tiempo. En este caso, el factor determinante fue la cantidad de bits que debían organizarse en bloques, codificarse, transmitirse y recuperarse.

Sin embargo, la Tabla 2 y la Figura 2 muestran que el nivel de ruido sí tuvo un impacto directo sobre la capacidad de recuperación. Con una probabilidad de 0.01 %, Hamming recuperó correctamente 497 de las 500 transmisiones que realmente presentaron flips, equivalente aproximadamente al 99.4 %. Con 0.1 %, corrigió 529 de 600 transmisiones, mientras que con 1 % solamente logró recuperar 5. Esto puede explicarse porque Hamming (12,8) corrige como máximo un bit por bloque de 12 bits y, conforme aumenta el ruido, también crece la posibilidad de que dos o más flips coincidan dentro del mismo bloque. Adicional, esta capacidad de corrección produjo una sobrecarga constante del 50 %, ya que se agregaron cuatro bits de paridad por cada ocho bits de datos (Yass, 2026a).

CRC-32 presentó una tendencia temporal similar. De acuerdo con la Tabla 4 y la Figura 3, la mediana aumentó desde aproximadamente 4.35 ms para una página

hasta un rango de 24 a 26 ms para las seis páginas. Las diferencias entre probabilidades continuaron siendo menores que las producidas por el tamaño, aunque en los mensajes más grandes las transmisiones con errores tendieron a finalizar un poco antes. Consideramos que esto puede deberse a que, cuando CRC-32 detectaba una alteración, el servidor respondía sin continuar con la recuperación y validación del contenido de ese ensayo. No obstante, esta explicación representa una inferencia sobre nuestra implementación y no una propiedad general del algoritmo.

La Tabla 5 y la Figura 4 muestran que CRC-32 detectó todas las transmisiones que realmente presentaron flips dentro de la muestra analizada. Por ejemplo, con 0.01 % se obtuvieron 195 transmisiones limpias y 405 alteradas, y las 405 fueron detectadas. Con 0.1 % y 1 %, las 600 transmisiones de cada grupo presentaron errores y todas fueron identificadas. Esto no significa que CRC-32 sea infalible ante cualquier patrón posible, pero sí que durante las pruebas no se presentó ninguna corrupción sin detectar. Su principal limitación fue que, después de identificar el error, no podía corregir el mensaje.

Al comparar la sobrecarga, CRC-32 presentó una ventaja considerable. Como se observa en la Tabla 6 y la Figura 5, se agregaron únicamente 32 bits a cada mensaje completo (Yass, 2026a), por lo que la sobrecarga disminuyó de 0.71 % a 0.12 % conforme aumentó el contenido. Hamming, por otro lado, mantuvo un 50 % de sobrecarga para todos los tamaños. A pesar de esta diferencia, Hamming obtuvo mejores tiempos dentro de nuestra implementación, alcanzando alrededor de 14.5 ms para seis páginas, frente a los 24–26 ms de CRC-32. Esto podría estar relacionado con la forma en que implementamos la división polinómica y las operaciones XOR de CRC-32, por lo que no sería correcto concluir que Hamming siempre es más rápido en cualquier sistema.

Dicho esto, no consideramos que exista un algoritmo superior para todos los escenarios. Hamming ofreció mejores tiempos y permitió continuar la comunicación cuando los errores se encontraron dentro de su capacidad de corrección, pero su efectividad cayó considerablemente con 1 % de ruido y generó mucha más redundancia. CRC-32, al contrario, presentó una sobrecarga mínima y detectó todas las alteraciones observadas, pero necesitaba descartar o retransmitir el mensaje porque no podía corregirlo.

Por último, utilizaríamos detección de errores cuando la retransmisión fuera posible y su costo fuera aceptable, especialmente en canales relativamente confiables o en protocolos donde se prefiera rechazar cualquier mensaje alterado. En cambio, utilizaríamos corrección cuando retransmitir fuera costoso, lento o imposible (Yass, 2026b), como podría ocurrir en comunicaciones unidireccionales o con alta latencia. Sin embargo, en un canal con demasiado ruido, Hamming (12,8) tampoco sería suficiente, debido a su límite de un error por bloque, por lo que sería necesario considerar un código de corrección más robusto. En nuestro caso, los resultados muestran que la elección depende del equilibrio buscado entre continuidad, redundancia, nivel de ruido y posibilidad de retransmisión.

Conclusiones

- La arquitectura por capas permitió separar claramente las responsabilidades del protocolo y comprobar que la comunicación es independiente del lenguaje de programación, ya que el cliente en Python y el servidor en Java lograron interactuar mediante las mismas reglas y el formato de mensajes establecido.
- El tamaño del mensaje fue el factor que más influyó en el tiempo de transmisión. Dentro de nuestra implementación, Hamming (12,8) presentó menores tiempos que CRC-32 y logró corregir transmisiones con poco ruido; sin embargo, su efectividad disminuyó cuando varios bits fueron alterados dentro de un mismo bloque.
- No existe un algoritmo superior para todos los escenarios. Hamming permite continuar la comunicación a cambio de una sobrecarga del 50 %, mientras que CRC-32 agrega únicamente 32 bits y detectó todas las alteraciones de la muestra, pero requiere descartar o retransmitir el mensaje. Por lo tanto, la elección depende del nivel de ruido, el costo de la redundancia y la posibilidad de retransmisión.

Referencias

- cnet128. (2015, 17 de abril). *One Piece 783: You're in my way* [Traducción al inglés]. MangaHelpers. <https://mangahelpers.com/t/cnet128/releases/41231>
- OpenAI. (2026). *ChatGPT* (modelo GPT-5.6 Sol) [Modelo de lenguaje de gran escala]. <https://chatgpt.com/>

- Yass, J. (2026a). *CC3067: Redes de computadoras: Capa de enlace I* [Diapositivas de clase]. Universidad del Valle de Guatemala.
- Yass, J. (2026b). *CC3067: Redes de computadoras: Modelos de referencia* [Diapositivas de clase]. Universidad del Valle de Guatemala.